

Google Hacking → finding security weaknesses in web applications

1). Directory Listing (exposing sensitive accessible file)

(web server feature where it shows the contents within that directories {e.g. index.html, style.css, etc.})

***Google Dorking** → using Google's advance search operators in google to find specific information (searching publicly accessible file)

e.g. `site:hacking-web-applications.com` → searching specific domain "hacking-web-applications.com"

e.g. `site:hacking-web-applications.com AND intitle:"Index of"` → **AND** : the search result must match both conditions

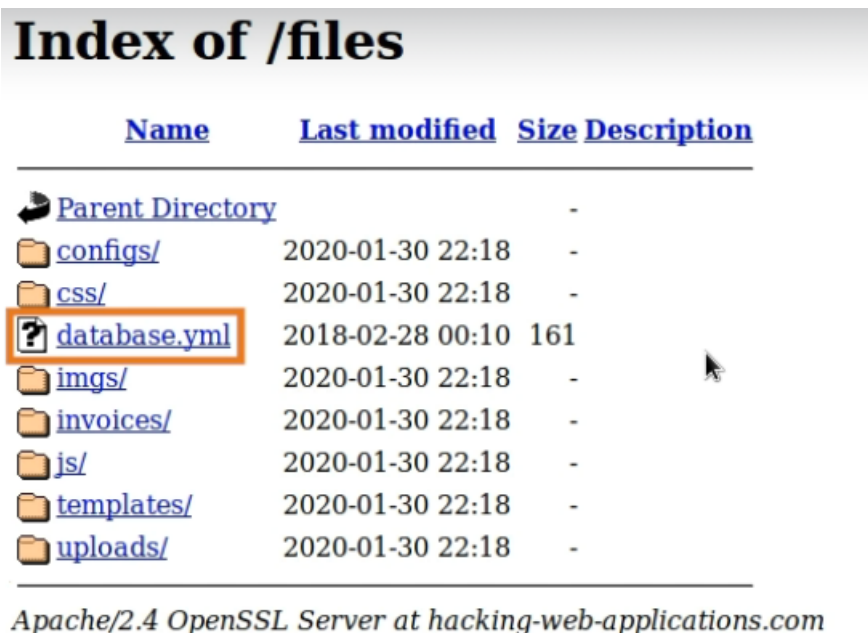
→ **intitle** : search for pages with

"Index of" in their title

1). after searching with the following queries, the target will appear in the search result



2). after pressing, the directory listing will also appear



3). pressing at the .yml file will result in exposing sensitive contents

```
1
2 production:
3   adapter: mysql2
4   host: example.com
5   port: 3306
6   username: root
7   password: supersecurepassword
8   database: supersecureshop
9   encoding: utf8
```

2). Finding SQL Syntax Errors

Most common SQL usage : Microsoft SQL, MySQL, PostgreSQL, Oracle Database, etc.

Search engine payload e.g. :

MySQL : `site:hacking-web-applications.com AND ("mysql error with query" OR "mysqli_connect" OR "pdo_mysql")`

Microsoft SQL : `site:hacking-web-applications.com AND ("[MICROSOFT][ODBC SQL " OR "[SQL SERVER]"`

PostgreSQL : `site:hacking-web-applications.com AND ("PostgreSQL server: FATAL" OR "not valid PostgreSQL result")`

Oracle : `site:hacking-web-applications.com AND ("ORA-00933:" OR "ORA-00921:" OR "ORA-12541" OR "[ODBC SQL")`

The second operator is to find the string related to the error base on the SQL the web server uses

1). After searching with the above queries, we need to try searching with different SQL (we need to match with the web server's SQL type)

Your search - `site:hacking-web-applications.com AND ("mysql error with query" OR "mysqli_connect" OR "mysqli_query" OR "pdo_mysql")` - did not match any documents.

Suggestions:

- Make sure all words are spelled correctly.
- Try different keywords.
- Try more general keywords.
- Try fewer keywords.

2). If it matches it will appear

hacking-web-applications.com > products > id ▼
[Microsoft][ODBC SQL Server Driver][SQL Server]Unclosed...
Server Error in '/' Application. [Microsoft][ODBC SQL Server Driver][SQL Server]Unclosed quotation mark after the character string "

3). After clicking it,

Server Error in '/' Application.

[Microsoft][ODBC SQL Server Driver][SQL Server]Unclosed quotation mark after the character string Mark'.

Description: An unhandled exception occurred during the execution of the current web request. Please review the stack trace for more information about the error and where it originated in the code.

Exception Details: System.Data.SqlClient.SqlException: Unclosed quotation mark after the character string Mark'.

Source Error:

```
Line 47:         using (SqlConnection connection = new SqlConnection("Data Source=sqls.local; Database = webapp1; User Id = sa; Password =  
Line 48:         using (SqlCommand cmd = new SqlCommand($"SELECT id, owner id, name, description, price FROM products WHERE name = '{pr  
Line 49:             connection.Open(); using (SqlDataReader reader = cmd.ExecuteReader())  
Line 50:         {  
Line 51:             if (!reader.HasRows)
```

Source File: C:\WebApplication\Controllers\StoreController.cs **Line:** 49

Stack Trace:

```
[SqlException (0x80131904): Unclosed quotation mark after the character string Mark'.]  
System.Data.SqlClient.SqlConnection.OnError(SqlException exception, Boolean breakConnection, Action`1 wrapCloseInAction) +3318656  
System.Data.SqlClient.TdsParser.ThrowExceptionAndWarning(TdsParserStateObject stateObj, Boolean callerHasConnectionLock, Boolean asyncC  
System.Data.SqlClient.TdsParser.TryRun(RunBehavior runBehavior, SqlCommand cmdHandler, SqlDataReader dataStream, BulkCopySimpleResultSet  
System.Data.SqlClient.SqlDataReader.TryConsumeMetaData() +89  
System.Data.SqlClient.SqlDataReader.get_MetaData() +101  
System.Data.SqlClient.SqlCommand.FinishExecuteReader(SqlDataReader ds, RunBehavior runBehavior, String resetOptionsString, Boolean isInt  
System.Data.SqlClient.SqlCommand.RunExecuteReaderTds(CommandBehavior cmdBehavior, RunBehavior runBehavior, Boolean returnStream, Boolean  
System.Data.SqlClient.SqlCommand.RunExecuteReader(CommandBehavior cmdBehavior, RunBehavior runBehavior, Boolean returnStream, String met  
System.Data.SqlClient.SqlCommand.RunExecuteReader(CommandBehavior cmdBehavior, RunBehavior runBehavior, Boolean returnStream, String met
```

4). We can find the credentials to the data base

```
using (SqlConnection connection = new SqlConnection("Data Source=sqls.local; Database = webapp1; User Id = sa; Password = tioNd39Iz6co"))  
using (SqlCommand cmd = new SqlCommand($"SELECT id, owner id, name, description, price FROM products WHERE name = '{product.name}', conne  
{  
    connection.Open(); using (SqlDataReader reader = cmd.ExecuteReader())  
    {  
        if (!reader.HasRows)
```

3). Publicly Exposed Back Up Files

payload: site:hacking-web-applications.com AND (ext:"backup" OR ext:"bak" OR ext:"old") ➡ ext is the extension we're looking for

1). After searching through the 'ext' parameter :

```
hacking-web-applications.com › backups ▾  
USE [master] GO CREATE DATABASE [webapp1] CONTAINMENT = NONE...  
USE [master] GO CREATE DATABASE [webapp1] CONTAINMENT = NONE ON PRIMARY(  
NAME = N'webapp1', FILENAME = N'C:\Program Files\Microsoft SQL...
```

2). After downloading the backup file,

```
1 USE [master]  
2 GO  
3 CREATE DATABASE [webapp1]  
4 CONTAINMENT = NONE  
5 ON PRIMARY  
6 ( NAME = N'webapp1', FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL15.NINJAMSSQL\MSSQL\DATA\webapp1.mdf' , SIZE = 8192KB , MAXSIZE = UNLIMITED,  
  FILEGROWTH = 65536KB )  
7 LOG ON  
8 ( NAME = N'webapp1_log', FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL15.NINJAMSSQL\MSSQL\DATA\webapp1_log.ldf' , SIZE = 8192KB , MAXSIZE =  
  2048GB , FILEGROWTH = 65536KB )  
9 WITH CATALOG_COLLATION = DATABASE_DEFAULT  
10 GO  
11 ALTER DATABASE [webapp1] SET COMPATIBILITY_LEVEL = 150  
12 GO  
13 IF (1 = FULLTEXTSERVICEPROPERTY('IsFullTextInstalled'))  
14 begin  
15 EXEC [webapp1].[dbo].[sp_fulltext_database] @action = 'enable'  
16 end  
17 GO  
18 ALTER DATABASE [webapp1] SET ANSI_NULL_DEFAULT OFF  
19 GO  
20 ALTER DATABASE [webapp1] SET ANSI_NULLS OFF  
21 GO  
22 ALTER DATABASE [webapp1] SET ANSI_PADDING OFF  
23 GO  
24 ALTER DATABASE [webapp1] SET ANSI_WARNINGS OFF  
25 GO  
26 ALTER DATABASE [webapp1] SET ARITHABORT OFF  
27 GO  
28 ALTER DATABASE [webapp1] SET AUTO_CLOSE OFF  
29 GO  
30 ALTER DATABASE [webapp1] SET AUTO_SHRINK OFF
```

3).After further scrolling, the admin email and password hash will also be visible (Don't forget to view it as a SQL syntax)

```
228 INSERT [dbo].[AspNetUsers] ([Id], [Email], [EmailConfirmed], [PasswordHash], [SecurityStamp], [PhoneNumber], [PhoneNumberConfirmed], [TwoFactorEnabled], [LockoutEndDateUtc], [LockoutEnabled], [AccessFailedCount], [UserName]) VALUES (N'ac4ec352-08de-49a5-a467-f9dc8395f71e', N'admin@example.com', 0, N'AFx0dIJaopnu4W3T4DgIGjW9I8BH6ENSfgpqCW3ciJEt/fIXM2L3HNM75jDJGRB8Ew=', N'4ac7de1d-8994-484e-9d9f-e8f4af5e0c73', NULL, 0, 0, NULL, 1, 0, N'admin@example.com')
```

4). Finding Internal Server Error

payload : `site:hacking-web-applications.com AND intitle:"500"`

- This 500 is for the http internal server error code, 500 is typically displayed when error occurred

1).After searching the queries

Warning: include(French.php): failed to open stream: No such file or directory in /var/www/index.php on line 31

#	Time	Memory	Function	Location
1	0.0000	356768	{main}()	.../index.php:0

Warning: include(): Failed opening 'French.php' for inclusion (include_path='./:/usr/share/php') in /var/www/index.php on line 31

#	Time	Memory	Function	Location
1	0.0000	356768	{main}()	.../index.php:0

2). Looking at the url there could appear a weak parameter in it

`https://hacking-web-applications.com/?language=French.php`

3). From the parameter accessing the admin/.htpasswd to access the credentials

`https://hacking-web-applications.com/?language=admin/.htpasswd`
`root:supersecurepassword`

5). Finding Sensitive Data in URL

payload : `site:hacking-web-applications.com AND (inurl:"api" OR inurl:"key" OR inurl:"apikey")` → finding API keys from the url

- inurl means to find the specific string in the url

1). After searching,

Google search results for the query: `site:hacking-web-applications.com AND (inurl:"api" OR inurl:"key" OR inurl:"apikey")`. The search returned 1 result in 0.15 seconds.

The result snippet shows a JSON array of user data from `hacking-web-applications.com > api > users`. The first user is:

```
{ "name": "Michael", "email": "michael@example.com", "password_hash": "0da30d90fe6a6a8c8f31f13b462daabb" }
```

The URL in the address bar is: `https://hacking-web-applications.com/api/show-all-users?apikey=21f9193f-2395-4087-be61-0df56573c7a1`

2). API key is to read people's email and the password hashes, knowing the password next is to crack the password (can use hashcat, etc.)

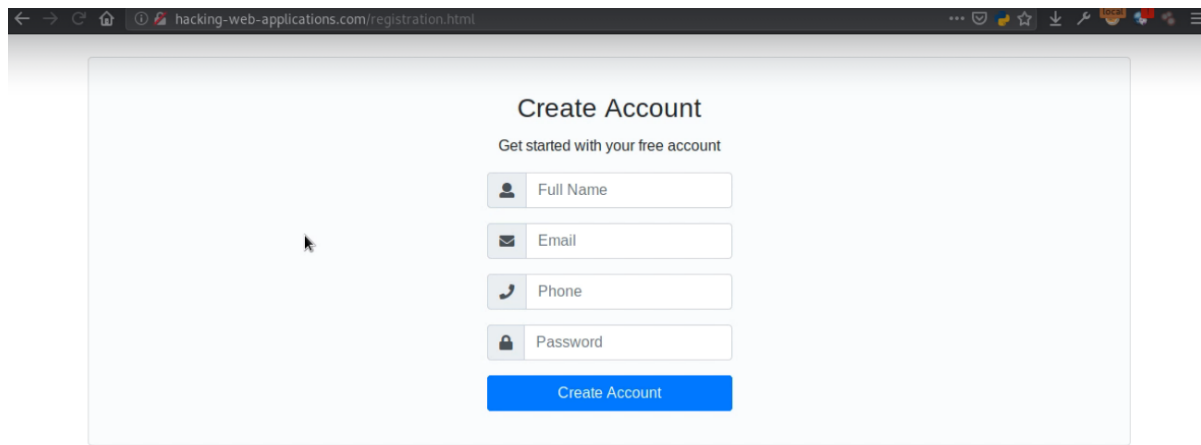
JSON viewer showing the response from the API. The data is an array of user objects:

```
[{"name": "Michael", "email": "michael@example.com", "password_hash": "0da30d90fe6a6a8c8f31f13b462daabb"}, {"name": "John", "email": "john@example.com", "password_hash": "bc25d16b25f3f7b918faace737a5fa8"}, {"name": "Emily", "email": "emily@example.com", "password_hash": "35e26ccdb161ed93dd2e9add4fc0d59e"}, {"name": "Alice", "email": "alice@example.com", "password_hash": "517cf4e82a3b50a83003a280ca4c9c2f"}, {"name": "Sophia", "email": "sophia@example.com", "password_hash": "833b1e4cc3c0efb545c85d097cb7d5d2"}, {"name": "Lucas", "email": "lucas@example.com", "password_hash": "ef4f6f3837f635aedf797c461da842e0"}]
```

6). Finding Insecure HTTP Web Pages

payload: `site:hacking-web-applications.com AND -inurl:"https"` → `"-inurl"`: exclude the https | finding the http (non secure ones)

1). After searching :



- it is a http web which is not secured

(This method is good for finding insecure web pages)

Google Hacking Database

Includes Google Search engine Queries (Google Dorks)

<https://www.exploit-db.com/google-hacking-database> (for more)

Prevent Getting Google Indexed

1). Adding robots.txt file in the root directory of your web-application :

```
User-agent: *  
Disallow: /
```

- this message telling google bots and other bots not to index the web pages